Coherencia no es completitud — el punto ciego de uv pip check

`uv pip check` mide coherencia, no completitud

Nota técnica · medición del 2026-11-03 · caso: `BerriAI/litellm`

---

§0 Abstract

Una instalación de Python puede pasar `uv pip check` y, doce segundos después, fallar al importar un paquete que no está instalado. No es un defecto de `uv pip check`: es un instrumento que responde una pregunta distinta de la que el pipeline cree estar haciendo. `uv pip check` mide **coherencia** —que lo instalado sea mutuamente compatible—; la pregunta que decide si un build pasa es **completitud** —que todo lo que se importa esté instalado—. Entre las dos hay un hueco que ningún instrumento del pipeline cierra por construcción: el único guardián de completitud es una lista de módulos escrita a mano, y su modo de fallo es el silencio. Se documenta un caso medido, con marcas de tiempo, y se formula el falsador que la afirmación tendría que sobrevivir.

---

§1 La afirmación

> **Un `uv pip check` verde no acredita que el grafo de imports resuelva.** Acredita que los paquetes ya instalados declaran requisitos compatibles entre sí. La diferencia no es teórica: en el caso medido, el verde y el rojo están separados por dos líneas del mismo `run:` y por 12,06 segundos del mismo entorno virtual.

La afirmación es de método, no de producto. No dice que `uv pip check` esté mal: dice que **un resultado verde de un instrumento que no mide lo que se necesita se lee como si lo midiera**, y esa lectura es la que deja pasar el defecto.

---

§2 El caso medido

`BerriAI/litellm` ejecuta el job `resolve` sobre seis versiones de Python. El paso que decide el job es `.github/workflows/test-mcp-dependency-resolution.yml:82-97`:

```
# :85
for extra in core mcp proxy; do
  # :88
  UV_PROJECT_ENVIRONMENT=".venv-$extra" ./uv_sync_with_retries.sh --frozen --no-dev --no-editable
        ...
  # :89
  uv pip check --python ".venv-$extra"
  ...
  # :96
  (cd "$RUNNER_TEMP" && ".../.venv-$extra/bin/python" "${checker[@]}")
done
```

En el run `41733107293`, job `resolve (3.12)` (id `126255925464`), el log del mismo paso, en el mismo entorno `.venv-proxy`, dice las dos cosas siguientes:

```
22:55:46.4830286Z  All installed packages are compatible     ← línea 814 del log
22:55:58.5402442Z  ModuleNotFoundError: No module named 'prisma'  ← línea 835
```

Los seis saltos intermedios, leídos del wheel instalado (no del árbol de fuentes, y no de memoria):

```
scripts/check_mcp_sdk_install.py:54   mcp_server.unsupported_protocol_version(scope)
  litellm/proxy/mcp_server/server.py:175
    litellm/proxy/mcp_server/capabilities.py:76   from litellm.proxy.proxy_server import …
      litellm/proxy/proxy_server.py:922           from litellm.proxy.moyai_endpoints import …
        litellm/proxy/moyai_endpoints.py:29       from …ui_crud_endpoints.proxy_setting_endpoints
import …
          litellm/proxy/ui_crud_endpoints/__init__.py:1
            litellm/proxy/ui_crud_endpoints/proxy_setting_endpoints.py:19
              from prisma.errors import PrismaError   → ModuleNotFoundError
```

La causa del hueco está a dos líneas de distancia en `pyproject.toml`:

```
:57    proxy = [ … ]          ← el extra que el workflow sincroniza
:112   extra_proxy = [
:113       "prisma>=0.11.0,<1.0",
```

`prisma` vive en `extra_proxy`; el bucle recorre `core mcp proxy`. El import es de alcance de módulo, así que basta con tocar `proxy_setting_endpoints` desde cualquier ruta transitiva para romper la instalación — y la ruta transitiva existe, y pasa por siete módulos que nada declara como dependencia de `proxy`.

---

§3 El mecanismo: dos grafos, un solo instrumento

Hay dos grafos en juego y un solo instrumento mirando:

grafo	qué afirma	quién lo mide
declarado	«este extra requiere estas distribuciones»	`uv pip check`, `uv lock --check`
importado	«este módulo importa estos nombres»	**una lista escrita a mano**

`uv pip check` recorre el grafo **declarado** de lo que ya está instalado. Un import cuyo nombre nunca se declaró en el extra activo no produce ninguna arista declarada, así que no hay nada que el instrumento pueda contradecir. Verdadero y vacío a la vez.

Del lado del grafo importado, el único guardián del pipeline es `IMPORTED_MODULES`, un literal en `scripts/check_mcp_sdk_install.py` que un humano edita. Su modo de fallo es el **silencio**: si un módulo no está en la lista, el chequeo no se queja — simplemente no lo mira. Un guardián que calla cuando le falta cobertura es indistinguible de un guardián que está conforme.

Por eso el defecto no lo encontró la medición de completitud: lo encontró el **accidente** de que la ruta transitiva del script enumerado atravesara el módulo culpable. Un import transitivo más corto, o un `registry` distinto, y el job habría pasado verde con la dependencia igual de ausente.

---

§4 Qué cerró el defecto y qué no

El defecto concreto está cerrado: `BerriAI/litellm#49771` (`fix(proxy): defer optional Prisma imports in SSO settings`), mergeado el `2026-11-03T20:10:16Z`, merge commit `200ecdb6e6700d502bf56f74131ee5253f11dc85`, 3 archivos, `+31 / −2`.

El remedio, leído del patch:

- `proxy_setting_endpoints.py`: `from prisma.errors import PrismaError` pasó a `if TYPE_CHECKING:` más tres imports locales en los sitios de uso.

- `check_mcp_sdk_install.py`: **una línea** — `"litellm.proxy.proxy_server"` añadido a `IMPORTED_MODULES`.

Es una decisión de diseño legítima (prisma no debe estar en el camino de import del proxy) y un remedio de método **manual**: el sistema se arregló ensanchando la lista escrita a mano. La pregunta de §3 sigue abierta. `#49771` es, al mismo tiempo, la prueba de que el hueco era real y la prueba de que hoy sólo se tapa de uno en uno.

---

§5 La forma general

El caso es de `litellm` por accidente de calendario, no por naturaleza. La propiedad es de cualquier pipeline que haga `pip check` / `uv pip check` como única validación de entorno:

1. Instrumenta el grafo **declarado**, que es el que el proyecto controla.
2. No instrumenta el grafo **importado**, que es el que Python ejecuta.
3. Compensa con listas de humo escritas a mano.
4. Y cuando la lista se queda corta, el resultado no es un rojo: es un verde con menos cobertura, que es peor que no medir.

Lo que cerraría el hueco, enunciado para que otro lo refute, es un instrumento que compare ambos grafos: la unión de módulos alcanzables desde los puntos de entrada declarados, contra las distribuciones instaladas en el entorno del extra, con `RC ≠ 0` cuando la diferencia no esté vacía.

---

§6 Falsadores

La nota se retira si ocurre cualquiera de estas tres cosas, y cada una se puede medir con un comando:

#	falsador	sonda
F-I	El pipeline de `litellm` ya tiene un instrumento que compare el grafo importado contra el instalado y emita `RC ≠ 0`.	`grep -rn "pip check|import.*graph|module.*graph" .github/workflows/` sobre `main`; si existe una etapa así, §3 es falsa.
F-II	`IMPORTED_MODULES` se **genera** desde el grafo real en vez de escribirse a mano.	`gh api repos/BerriAI/litellm/contents/scripts/check_mcp_sdk_install.py` — hoy es un literal (medido 2026-11-03).
F-III	Un entorno que pasa `uv pip check` cubre **por construcción** todo import de alcance de módulo alcanzable desde sus puntos de entrada.	Construir un entorno con los extras declarados que omita una distribución importada en alcance de módulo y observar el verde. Es el caso de §2; para refutar la generalización basta uno en que **no** ocurra.

Nótese lo que **no** es falsador: que `uv pip check` haga bien su trabajo. Eso es el supuesto de la nota, no su objeción.

---

§7 Lo que esta nota no afirma

- **No** afirma que `litellm` esté roto: el defecto concreto está cerrado (`#49771`) y el proyecto respondió en menos de un día.
- **No** afirma que `uv pip check` tenga un defecto: cumple lo que documenta.

- **No** afirma que `prisma` deba estar en el extra `proxy`: los mantenedores eligieron imports diferidos, y es una elección defendible.
- **No** afirma que el job `resolve` esté mal diseñado: el job encontró el defecto. Afirma que lo encontró **por accidente**, y que el accidente no se puede volver a invocar a voluntad.
- **No** mide prevalencia: `n = 1` caso, bien trazado. Un `n = 1` no sostiene una tasa, sostiene una existencia.

---

§8 Bloque de depósito (Zenodo)

Listo para depositar; el depósito **no** lo ejecuta esta nota.

```
Título        : uv pip check mide coherencia, no completitud
Subtítulo     : Un caso medido en CI de Python: verde de dependencias,
                import roto, 12,06 s de distancia
Tipo          : Publication / Technical note
Fecha         : 2026-11-03
Autor         : (resolver vía ORCID_ID del entorno — tools/meta_env.py --nombres)
Licencia      : CC-BY-4.0
Idioma        : spa
Keywords      : uv, pip check, dependency resolution, import graph,
                CI reliability, Python packaging, extras, silent green
Relacionado   : url  https://github.com/BerriAI/litellm/pull/49771
                (isSupplementedBy — el remedio medido)
                url  https://github.com/BerriAI/litellm/actions/runs/41733107293
                (isDerivedFrom — la evidencia cruda)
```

KPI del depósito, declarado antes de depositar: `cited_by_count > 0` en OpenAlex y descargas acumuladas, medidos por `tools/medir_adopcion.py`. El número de DOIs no es el KPI: un cuarto DOI con la misma distribución mide cero. La línea base de hoy —0 citas en los tres DOIs existentes— está medida en `VISION_OPEN_SOURCE.md` §1 y no se mueve por publicar esta nota.

Frontera. La política de comentar en repos ajenos (los 3 filtros) **no** vive aquí: vive en `VISION_OPEN_SOURCE.md` §3. Este documento la usa, no la define.